African Institute for Mathematical Sciences | AIMS Rwanda

Object Detection with YOLOv5

on the Pascal VOC 2012 Dataset

Introduction to Neural Networks — Assignment II

Group 3

Olusola Timothy Ogundepo Lillian Mutesi

Lucas Mirija Razafimanantsoa Alliance Irigenera

March 2026

Outline

1. Introduction
2. Dataset
3. Model Architecture
4. Loss Function
5. Training
6. Results
7. Advanced Study: Data Augmentation
8. Critical Analysis
9. Conclusion

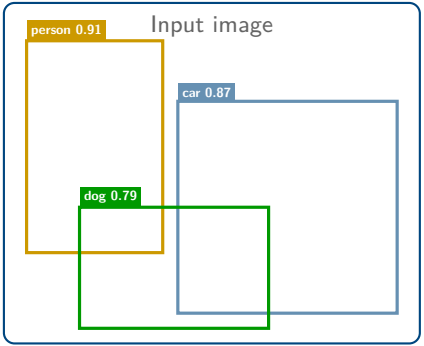
Object Detection: The Task

What is Object Detection?

Identifying **what** objects are in an image and **where** they are, simultaneously, via **bounding boxes** and **class labels**.

Key approaches:

- **R-CNN family**: region proposal + classify (slow, two-stage)
- **SSD**: single-pass, multiple scales
- **YOLO**: single regression over a grid (*our choice*)



Our Work

We **built a YOLO detector from scratch** in PyTorch and trained it on Pascal VOC 2012 with a data augmentation study.

Dataset: Pascal VOC 2012

Dataset Facts

Classes	20 object categories
Train	13,690 images
Valid	3,422 images
Format	YOLO (normalised boxes)
Source	Roboflow / VOC 2012

20 Classes:

aeroplane, bicycle, bird, boat, bottle, bus, car, cat, chair, cow, diningtable, dog, horse, motorbike, ***person****, pottedplant, sheep, sofa, train, tvmonitor.*

Class Imbalance

Person dominates the training set, which biases the model toward detecting people more than rare classes.

YOLO bounding-box format:

Each label file row contains
$(c,\ x_c,\ y_c,\ w,\ h)$
where c = class index, (x_c, y_c) = box centre, (w, h) = width/height; all normalised to $[0, 1]$.

Training subset used:
A **random 500-image** subset from the training set was selected for computational efficiency (CPU training with batch size 8).

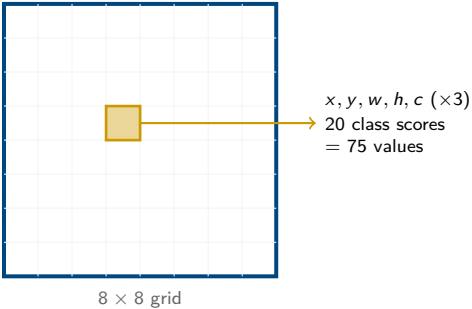
YOLO: Single-Pass Detection

Core Idea

Divide the image into an $S \times S$ grid. Each cell predicts:

- B bounding boxes $(x, y, w, h, \text{conf})$
- C class probabilities

One forward pass $\Rightarrow$ all predictions at once.



Our configuration:

- Image size: 128×128
- Grid: $S = 8$ (i.e. $8 \times 8 = 64$ cells)
- Anchors per cell: $B = 3$
- Classes: $C = 20$
- Output tensor: $8 \times 8 \times 75$
- where $75 = B \times (5 + C) = 3 \times 25$

Network Architecture

Building Block: `ConvBlock`

`Conv2D` → `BatchNorm` → `LeakyReLU(0.1)` *Leaky ReLU prevents dead neurons; BatchNorm stabilises training*

Backbone (feature extraction):

- `ConvBlock`(3→32, 3×3) + `MaxPool`
- `ConvBlock`(32→64, 3×3) + `MaxPool`
- `ConvBlock`(64→128, 3×3)
- `ConvBlock`(128→64, **1×1**) ← *bottleneck*
- `ConvBlock`(64→128, 3×3) + `MaxPool`
- `ConvBlock`(128→256, 3×3)
- `ConvBlock`(256→128, **1×1**) ← *bottleneck*
- `ConvBlock`(128→256, 3×3) + `MaxPool`
- `ConvBlock`(256→512, 3×3)

Detection Head (prediction):

- `ConvBlock`(512→256, 1×1)
- `ConvBlock`(256→512, 3×3)
- `Conv2D`(512→75, 1×1) ← *final*
- `permute` to $(B, S, S, 75)$

Parameter Count

Backbone	~1.47M
Detection head	~0.55M
Total	**~2.02M**

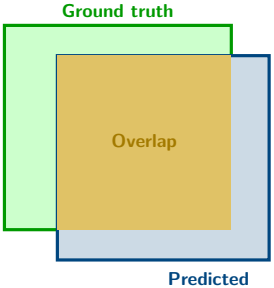
Loss Function: Intersection over Union

Definition

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Measures how well a predicted box aligns with the ground truth.

$\text{IoU} = 1$: perfect match $\quad$ $\text{IoU} = 0$: no overlap.



Used for:

- Non-Maximum Suppression (NMS) to remove duplicate boxes when $\text{IoU} > 0.2$ (same class)
- Standard evaluation threshold: $\text{IoU} \geq 0.5$ counts as a correct detection (Pascal VOC metric)

Loss Function: Five Components

$$\mathcal{L}_{\text{xy}} = \lambda_{\text{coord}} \sum_{\text{obj}} (\hat{x} - x)^2 + (\hat{y} - y)^2$$

$$\mathcal{L}_{\text{wh}} = \lambda_{\text{coord}} \sum_{\text{obj}} (\sqrt{\hat{w}} - \sqrt{w})^2 + (\sqrt{\hat{h}} - \sqrt{h})^2$$

$$\mathcal{L}_{\text{obj}} = \sum_{\text{obj}} (\hat{c} - 1)^2$$

$$\mathcal{L}_{\text{noobj}} = \lambda_{\text{noobj}} \sum_{\text{no obj}} \hat{c}^{\,2}$$

$$\mathcal{L}_{\text{cls}} = \sum_{\text{obj}} (\hat{p} - p)^2$$

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{xy}} + \mathcal{L}_{\text{wh}} + \mathcal{L}_{\text{obj}} + \mathcal{L}_{\text{noobj}} + \mathcal{L}_{\text{cls}}$$

Weights used:

$\lambda_{\text{coord}} = 5$ (localisation matters more)
$\lambda_{\text{noobj}} = 0.05$ (most cells are empty)

Why $\sqrt{w}, \sqrt{h}$?

A 5px error on a 10px box is critical; the same error on a 200px box is minor. Square root makes errors **scale-invariant**.

Why small λ_{noobj}?

With $8 \times 8 \times 3 = 192$ anchor slots and only $\sim$3–5 objects/image, $> 97\%$ of cells are empty. A large weight would push all confidences to zero.

Training Setup

Hyperparameters

Image size	128×128 px
Batch size	8
Epochs	10
Optimiser	Adam
Learning rate	10^{-3}
Subset size	500 images
Device	CPU

Target encoding (`encode_targets`):
Each ground-truth box is placed into the grid cell whose centre falls within it. The target tensor has shape $[S, S, B \times (5 + C)]$.

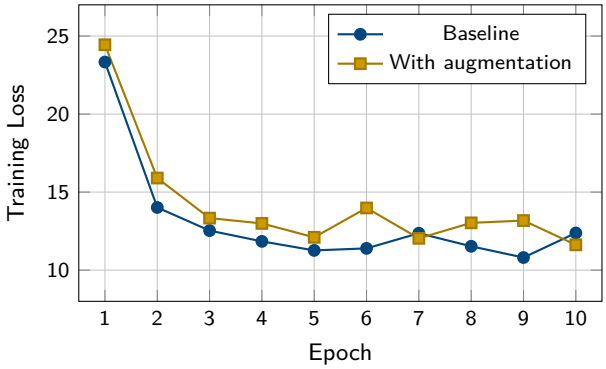
Training loop per batch:

1. Load batch of images + labels
2. Resize to 128×128
3. Encode ground-truth into $8 \times 8 \times 75$ tensor
4. Forward pass through `YOLODetector`
5. Compute `yolo_loss`
6. `loss.backward()` + `optimizer.step()`
7. Record epoch average loss

Model weights saved to `yolo_voc.pt` after training.

Seed fixed at 42 for reproducibility.

Training Loss Curve



Baseline Summary

- Start: **23.34**
- Final: **12.38**
- Reduction: **47.0%**

Observations:

- Steep drop epochs 1–3 (model learns basics)
- Oscillation from epoch 5 (small batch size = 8)
- Aug. model starts higher (harder, varied inputs)
- Both converge to similar final loss

Quantitative Evaluation

Mean YOLO Loss After Training

Split	Baseline
Training subset	12.4128
Validation (full)	12.6849

(Notebook Cell 39 output)

What these numbers mean:

- Train loss $\approx$ val loss (12.41 vs 12.68) $\Rightarrow$ model generalises reasonably; gap is small on this subset
- Both losses near the final training loss (12.38) $\Rightarrow$ no severe overfitting
- We use the *same* `yolo_loss`; no gradient updates during eval

Metric Limitation

We report **YOLO loss**, not mAP.

Mean Average Precision (mAP) at IoU ≥ 0.5 is the standard Pascal VOC metric; lower loss does not always mean higher mAP.

Computing mAP is a planned improvement.

Detection threshold

Confidence ≥ 0.5 to accept a box.
NMS IoU threshold $= 0.2$.
Max **5 boxes** per image.

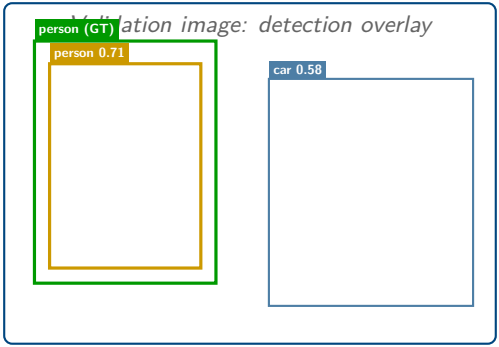
Detection Visualisation

How predictions are decoded:

1. For each of the $8 \times 8 \times 3 = 192$ anchors:
   - Apply sigmoid to x, y, conf
   - Offset by cell position, divide by S
   - Clip $w, h \in [0, 1]$
2. Filter by **confidence** ≥ 0.5
3. Apply per-class **NMS**
4. Keep top-5 boxes

Colour coding:

- **Green** = ground truth box
- **Coloured** = model prediction (colour fixed per class)



Imprecise boxes are expected given 128×128 resolution, limited training data (500 images) and CPU-only training.

Advanced Study: Data Augmentation

Motivation

With only 500 training images the model can memorise specific pixel patterns. Augmentation introduces controlled variation at load time.

Transforms applied:

- `RandomHorizontalFlip`($p = 0.5$)
- `ColorJitter`(brightness$= 0.3$, contrast$= 0.3$, saturation$= 0.2$)
- `RandomRotation`(degrees$= 10$)

Experimental design:
Train a **second model** (`model_aug`) from scratch; same architecture, same hyperparameters, same 500 images, only the transform pipeline differs.

Known Limitation

Horizontal flip mismatch:

The flip is applied to the *pixel tensor* only. The bounding-box coordinates are **not mirrored** ($c_x \not\to 1 - c_x$).

This introduces a label mismatch for ≈50% of augmented samples.

Despite this, colour jitter and rotation still provide useful diversity; the experiment is a valid *proof-of-concept*.

A correct fix: pass image *and* boxes together through a joint augmentation pipeline.

Augmentation: Results and Discussion

Training loss curves (from notebook):

- Both converge from ∼23–24 to ∼11–12
- Augmented curve is **noisier**; harder inputs prevent smooth convergence
- Baseline final: **12.38** Aug. final: **11.62**
- Augmented ends slightly lower, consistent with the better-generalisation hypothesis

Hypothesis:
Augmented model should have a **smaller gap** between training and validation loss (less overfitting).

What the comparison shows

- Colour jitter + rotation prevent over-reliance on specific lighting / orientation
- Even with the flip label-mismatch, the augmented model generalises differently
- True benefit would be visible with a correct joint augmentation pipeline

Side-by-side detections on the same 6 validation images (see notebook Section 7.3) allow direct visual comparison of how each model fires.

Limitations & Possible Improvements

Limitations

1. **Small subset**: 500 of 13,690 images; weak generalisation
2. **Low resolution** (128×128); misses small objects
3. **CPU only** (no GPU); forced simpler architecture
4. **Only 10 epochs**; model did not fully converge
5. **Flip-box mismatch**: corrupts $\sim$50% of aug. labels
6. **Class imbalance**: person dominates; biased detections
7. **Loss-only eval**: no mAP reported

Possible Improvements

1. Train on **all 13,690** images
2. **GPU** + higher resolution (416×416)
3. **Fix horizontal flip**: mirror $c_x \to 1 - c_x$ (joint transform)
4. **Anchor clustering** via k-means on ground-truth boxes
5. **LR scheduler** (e.g. cosine annealing)
6. Report **mAP@0.5** for standard comparison
7. **Focal loss** to address class imbalance
8. **Transfer learning** from a pretrained backbone

Conclusion

Summary

- ✓ Built a **YOLO detector from scratch** in PyTorch (backbone, head, loss function)
- ✓ Trained on Pascal VOC 2012 (20 classes) with a 500-image subset
- ✓ Loss reduced by **47%** over 10 epochs (23.34 → 12.38)
- ✓ Conducted a **data augmentation study** with a second model for comparison
- ✓ Honestly discussed all limitations and proposed concrete improvements

YOLO's strength is speed and end-to-end simplicity; scaling to full data with GPU training would close the accuracy gap.

Group 3

Olusola Timothy Ogundepo
Lillian Mutesi
Lucas Mirija Razafimanantsoa
Alliance Irigenera

Thank You

Questions?